SO/SO1/OS Project Description (2026)

***NEW:** Phase 2 requirements were added*

Build a C program in the UNIX environment implementing a **city infrastructure issue reporting and monitoring system**. City inspectors file structured reports about problems found across urban districts (damaged roads, broken lighting, flooding, etc.). The system stores and organises these reports on disk, enforces access rules based on Unix file permissions and user roles, monitors districts through a background process coordinated via signals, and routes data between components using pipes and I/O redirection with `dup2`.

Each phase builds upon the previous one. You must present your program at each phase. Not implementing a phase disqualifies the entire project and the student will not get a passing grade.

Phase 1: File Systems (Weeks 6–7)

Goal: Build the foundation for storing, querying, and managing infrastructure reports. Practice system calls for binary file I/O, file metadata, symbolic links, programmatic permission management, and AI-assisted filter implementation.

User Roles

The system recognises two roles: **inspector** and **manager**. Since students work on shared machines where creating Unix users is not possible, the role is passed explicitly on every command line invocation:

city_manager --role inspector --add downtown

city_manager --role manager --remove_report downtown 17

Directory and File Layout

Each district corresponds to a separate directory. Inside each district directory there is:

- A **binary report file** (`reports.dat`) storing fixed-size records, each containing at least:
    - Report ID (integer)
    - Inspector name (fixed-length string, provided as a `--user` argument)
    - GPS coordinates (latitude and longitude as floating-point numbers)
    - Issue category (fixed-length string, e.g. `"road"`, `"lighting"`, `"flooding"`)
    - Severity level (integer: 1 = minor, 2 = moderate, 3 = critical)
    - Timestamp (`time_t`)

    - Description text (fixed-length string)
- A plain-text **configuration file** (`district.cfg`) storing at least a severity threshold — the minimum level that should trigger an escalation alert
- A plain-text **operation log** (`logged_district`) recording every action performed on that district, with timestamp, declared role, and declared user name

Permission Bits

Managers are considered the owner users of the files. Inspectors are users in the group owning the file. Even though a single Unix user runs all commands, the program must create files with specific permission bits and check them before acting, using `chmod()`, `stat()`, and `lstat()`. The required permissions are:

File	Permissions	Meaning in this simulation
District directory	`rwxr-x---` (750)	Managers have full access; inspectors read and execute only
`reports.dat`	`rw-rw-r--` (664)	Both roles may read; both may write (append)
`district.cfg`	`rw-r-----` (640)	Managers may read and write; inspectors may read
`logged_district`	`rw-r--r--` (644)	Anyone may read; only the manager role may write

The program must set these permissions explicitly with `chmod()` when creating each file or directory. Before any operation that is role-restricted, it must call `stat()` on the relevant file, extract the permission bits from `st_mode`, and check them — printing a clear error and aborting if the declared role would not have the required access under the scheme above. For example, before reading `district.cfg`, check that the owner-read bit is set; before writing to `logged_district` as an inspector, detect the restriction and refuse.

The `list` command must always print the current permission bits of `reports.dat` in symbolic form (e.g. `rw-rw-r--`) alongside the file size and last modification time. Students must write the bit-to-symbol conversion themselves, not use an external tool.

Operations (commands)

`add <district_id>` Append a new report. Both roles may add reports. The inspector name field is taken from the `--user` argument. Set permissions on `reports.dat` to `664` after creation if not already set.

`list <district_id>` List all reports in a district, and the report file information in the format mentioned above.

`view <district_id> <report_id>` Print the full details of a specific report. Available to both roles.

`remove_report <district_id> <report_id>` Remove a single report. Manager role only. Use `lseek()` to shift subsequent records one position earlier, then truncate the file with `ftruncate()`.

`update_threshold <district_id> <value>` Update the severity threshold in `district.cfg`. Manager role only. Before writing, call `stat()` on `district.cfg` and verify the permission bits match `640`; if someone has changed them, refuse and print a diagnostic.

`filter <district_id> <condition>` Filter and display reports matching a condition. See the section below.

Example command lines:

city_manager --role manager --user alice --add downtown

city_manager --role inspector --user bob --list downtown

city_manager --role manager --user alice --remove_report downtown 17

city_manager --role inspector --user bob --filter downtown severity:>=:2 category:==:road

The `filter` Command and AI-Assisted Condition Matching

The `filter` command accepts one (or optionally more) condition(s). If more conditions are supported, they are given as distinct arguments separated by spaces. The command prints all reports that satisfy **all** of them (conditions are implicitly joined by AND). A condition is a single string of the form:

field:operator:value

Supported fields: `severity`, `category`, `inspector`, `timestamp`. Supported operators: `==`, `!=`, `<`, `<=`, `>`, `>=`.

What you are expected to do with AI assistance:

Use an AI assistant to help you implement two functions:

1. Describe your record structure and ask the AI to generate a function

```
int parse_condition(const char *input, char *field, char *op, char *value);
```

which splits a `field:operator:value` string into its three parts.

2. Describe the fields and their types and ask the AI to generate a function

```
int match_condition(Report *r, const char *field, const char *op, const char *value);
```

which returns 1 if the record satisfies the condition and 0 otherwise.

3. Review both functions line by line. You must be able to explain them fully at your presentation.
4. Write the `filter` logic yourself: open `reports.dat`, read records one by one with `read()`, parse each command-line condition with `parse_condition()`, test each record against every condition with `match_condition()`, and print records where all conditions return 1.
5. Describe in an `ai_usage.md` file the tool used, the prompts you gave, what was generated, what you changed and why, and what you learned. This file is a required deliverable and will be discussed at your presentation.

Symbolic Links and File Types

For each district, create a symbolic link in the root of the running program named `active_reports-<district_id>` pointing directly to that district's `reports.dat` file. Use `lstat()` — not `stat()` — whenever scanning directory contents, so symlinks are identified as such rather than followed silently. Detect and report dangling links (links to nonexistent files) with a warning rather than a crash.

System Calls to Use

`open()`, `close()`, `read()`, `write()`, `lseek()`, `ftruncate()`, `stat()`, `lstat()`, `mkdir()`, `symlink()`, `unlink()`, `chmod()`

Deliverables for Phase 1

- A working `city_manager` program in C implementing all operations above
- Correct creation and checking of permission bits as described
- A functional `filter` command using the AI-assisted condition matching functions
- The `ai_usage.md` file documenting your AI interaction and your critical evaluation of its output
- The `active_reports-*` symlinks correctly created, updated, and cleaned up

All deliverables should be submitted on Campus Virtual, until the last day of this Phase. Not submitting will lead to failing the entire project.

The code must be written by the student (with the AI-assisted exception above). In the evaluation period, students will have to explain in detail all the functionality, and answer related questions.

Hints for Phase 1

- To check whether a role has access, extract the relevant bits from `st_mode` using macros such as `S_IRUSR`, `S_IWGRP`, etc. and compare them against what your permission table requires. Do not compare the entire mode word — isolate only the bits you need.
- Write the permission-to-string conversion as a small helper function that checks each bit individually and builds a 9-character string like `rw-rw-r--`. This will also help you understand the bit layout of `st_mode` for the rest of the project.
- Test `remove_report` at the first, middle, and last record positions to make sure the shift logic is correct in all cases, and always check the file size before and after with `stat()`.
- When integrating `match_condition()`, the `value` string from the condition must be converted to the right C type before comparison — integer for `severity` and `timestamp`, string for `category` and `inspector`. Check whether the AI-generated function handles this correctly or leaves it to you.
- In `ai_usage.md`, be specific about what the AI got wrong or what you had to fix. This is not penalised — it is the most valuable gain of the AI-assisted exercise.

Phase 2: Processes and Signals (Weeks 8–9)

Modify the existing application as follows.

1. Add a new command to the existing `city_manager` program,
   `remove_district <district_id>`

that deletes the entire district directory and all its contents, plus the corresponding `active_reports-*` symlink. For deleting the directory, the program creates a child process, in which it calls the external command **`rm -rf <district_directory>`**. Be very careful when you test this so that you don't pass wrong arguments to **`rm`** thus deleting useful files or folders on your computer! The command is manager role only.

2. Write a new program called `monitor_reports` that works as follows:
   - at startup it creates or overwrites a hidden text file called **`.monitor_pid`** where it stores its main process ID. The file is situated at the same level of the directory tree as the district directories.
   - when it ends, it deletes the above file

- the program only ends when it receives SIGINT, and writes a message on the standard output when it does
- the program responds to SIGUSR1 signals (which informs it that a new report has been added) by writing a message on the standard output

3. Modify the existing `city_manager` program so that, when adding a new report, it notifies the monitor program of the event (by SIGUSR1), and writes a message acknowledging the notification in the corresponding log file for the report. If it doesn't find a PID of the monitor in the **`.monitor_pid`** file, or if any other error occurs (*e.g.,* sending the signal did not work), the message in the log file must explicitly show that the monitor could not be informed of the event.

System Calls to Use

`open()`, `close()`, `read()`, `write()`, `unlink()`, `sigaction()`, `kill()`, `fork()`, the `exec*()` family of system/library calls, other system calls that may be needed in the programs. Do **not** use the `signal()` call.

Deliverables for Phase 2

- The `city_manager` program modified as required above (C source). The features required in Phase 1 must continue to work properly. **Make sure they are all complete and that they work as required** (update your code if necessary)
- The `monitor_reports` program implemented as required above (C source)
- An `AI_usage-phases_1_and_2.md` file that documents in detail where, why and how have you used AI tools in developing the application in **both** Phases. You may include the text you wrote in the previous document for Phase 1 if necessary.
- The file and directory structure containing the complete outcome of what the updated program does, with at least 2 districts and 5 total reports.

Rules for submitting the system programming lab assignments:

- each student must have a **git repository** and weekly commits;
- each Phase will end with a mandatory code submission in a special Milestone assignment on Campus Virtual, which will include the other required deliverables
- failing to submit work for <u>any</u> of the phases **will void the entire project** (grade 2 at the lab for the project)

Help and even more hints

TLDR for Phase 1

An AI-generated TLDR version for Phase 1
(nevertheless, you still have to read the full spec!):

SO/SO1/OS Project – Quick Version 🏙️

You're building a **C program in UNIX** that manages city infrastructure reports (potholes, broken lights, floods, etc.) across districts. Three phases, each one due at the lab. **Miss one phase = fail the whole project. No exceptions.**

Phase 1 – Files (Weeks 6–7)

Build a tool called `city_manager` that stores reports in **binary files**.

You need to handle:

- Two roles: `inspector` and `manager`, passed on the command line
- A directory per district, each with a binary report file, a config file, and an operation log
- File permissions — you set them with code and check them before every operation
- Symbolic links pointing to each district's report file
- A `filter` command that searches reports by field (e.g. `severity:>=:2`)

The one place AI is officially allowed: two helper functions for the `filter` command. You must document exactly what you asked, what the AI gave you, and what you changed — in a file called `ai_usage.md`. You will be asked about this at the presentation.

You will not survive the presentation if you didn't write your own code.** They ~~will~~ may even open your binary file in a hex editor and ask you to point to specific bytes.

The rules, short version

- Weekly commits on Git — not optional
- Submit each phase on Campus Virtual by the deadline
- Show up and explain your code out loud
- No submission at any phase = grade 2 for the whole project 🪦

**That's a joke, obviously; never take what AI generates as literal truth. AI jokes are sometimes indistinguishable from Dad jokes.

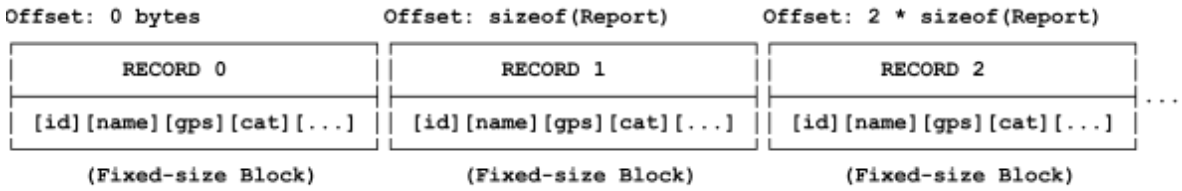
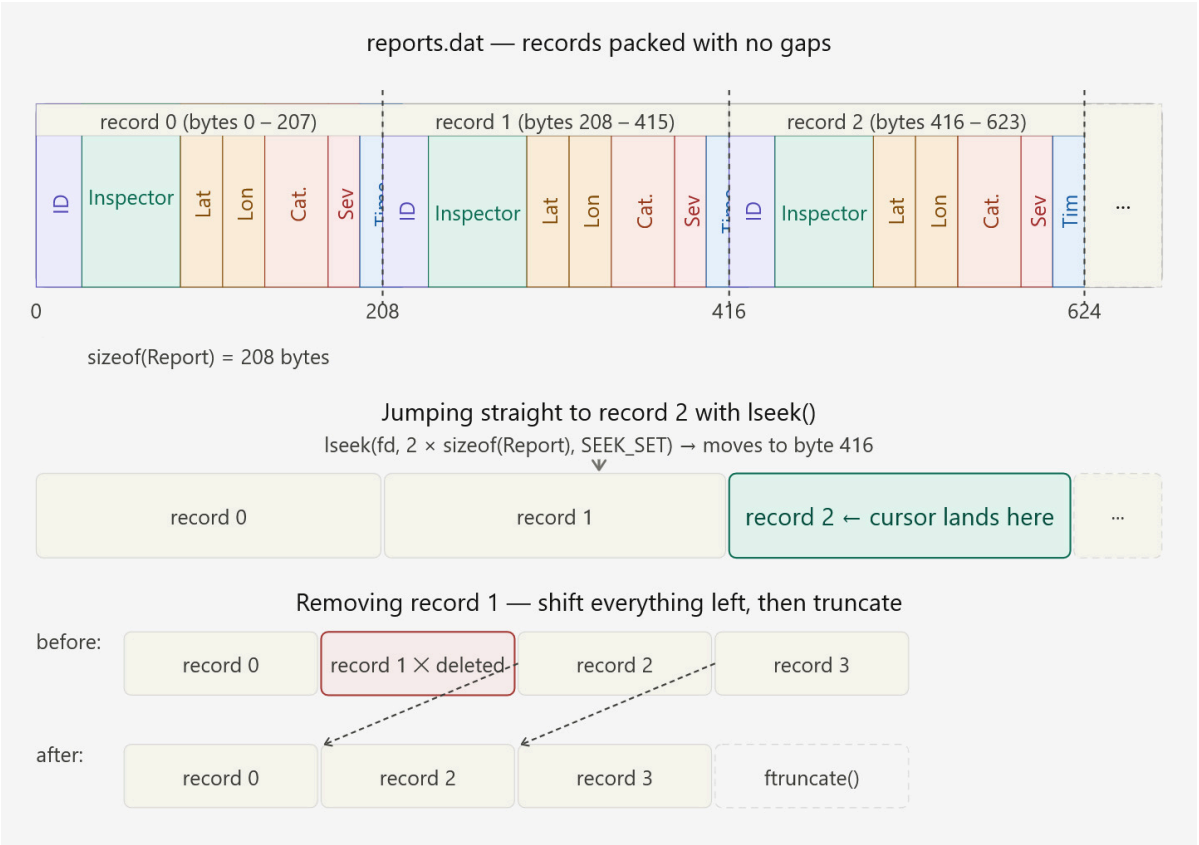
Files and directories

1. Example of file layout, command invocation and binary file content:

```
roland@ThinkpadT14:~/work/LabSO/Proiect$ ls -lh
total 12K
-rw-r--r-- 1 roland roland 8.2K Apr 15 23:31 city_manager.c
roland@ThinkpadT14:~/work/LabSO/Proiect$ gcc ./city_manager.c -o city_manager
roland@ThinkpadT14:~/work/LabSO/Proiect$ ls -lh
total 36K
-rwxr-xr-x 1 roland roland  22K Apr 15 23:32 city_manager
-rw-r--r-- 1 roland roland 8.2K Apr 15 23:31 city_manager.c
roland@ThinkpadT14:~/work/LabSO/Proiect$ ./city_manager --role manager --user alice --add downtown
X: 12.2
Y: 21.1
Category (road/lighting/flooding/other): road
Severity level (1/2/3):2
Description:Road closed
roland@ThinkpadT14:~/work/LabSO/Proiect$ tree -L 2
.
├── city_manager
├── city_manager.c
└── downtown
    ├── district.cfg
    ├── logged_district
    └── reports.dat

2 directories, 5 files
roland@ThinkpadT14:~/work/LabSO/Proiect$ cat ./downtown/logged_district
1776285168      alice   manager add
roland@ThinkpadT14:~/work/LabSO/Proiect$ cat ./downtown/district.cfg
roland@ThinkpadT14:~/work/LabSO/Proiect$ xxd ./downtown/reports.dat
00000000: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 0000 0000 0000 0000 6666 6666 6666 2840  ........ffffff(@
00000050: 9a99 9999 9919 3540 726f 6164 0a00 0000  ......5@road....
00000060: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000070: 0000 0000 0000 0000 0200 0000 0000 0000  ................
00000080: f0f5 df69 0000 0000 526f 6164 2063 6c6f  ...i....Road clo
00000090: 7365 6400 0000 0000 0000 0000 0000 0000  sed.............
000000a0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000b0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000c0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000d0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000e0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000f0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000100: 0000 0000 0000 0000                      ........
roland@ThinkpadT14:~/work/LabSO/Proiect$ ./city_manager --role manager --user alice --add Balcescu
X: 91.3
Y: 11.5
Category (road/lighting/flooding/other): lighting
Severity level (1/2/3):3
Description:No lights
roland@ThinkpadT14:~/work/LabSO/Proiect$ tree -L 2
.
├── Balcescu
│   ├── district.cfg
│   ├── logged_district
│   └── reports.dat
├── city_manager
├── city_manager.c
└── downtown
    ├── district.cfg
    ├── logged_district
    └── reports.dat

3 directories, 8 files
roland@ThinkpadT14:~/work/LabSO/Proiect$ ./city_manager --role manager --user alice --add downtown
X: 10.0
Y: 11.1
Category (road/lighting/flooding/other): road
Severity level (1/2/3):3
Description:Pothole in the middle of the road
roland@ThinkpadT14:~/work/LabSO/Proiect$ tree -L 2
.
├── Balcescu
│   ├── district.cfg
│   ├── logged_district
│   └── reports.dat
├── city_manager
├── city_manager.c
└── downtown
    ├── district.cfg
    ├── logged_district
    └── reports.dat

3 directories, 8 files
roland@ThinkpadT14:~/work/LabSO/Proiect$ cat ./downtown/logged_district
1776285168      alice   manager add
1776285269      alice   manager add
roland@ThinkpadT14:~/work/LabSO/Proiect$ xxd ./downtown/reports.dat
00000000: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 0000 0000 0000 0000 6666 6666 6666 2840  ........ffffff(@
00000050: 9a99 9999 9919 3540 726f 6164 0a00 0000  ......5@road....
00000060: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000070: 0000 0000 0000 0000 0200 0000 0000 0000  ................
00000080: f0f5 df69 0000 0000 526f 6164 2063 6c6f  ...i....Road clo
00000090: 7365 6400 0000 0000 0000 0000 0000 0000  sed.............
000000a0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000b0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
```

2. Visualizing the internal structure of the `reports.dat` binary file: